Classes, Objects, and Methods

A gentle introduction to object-oriented Python for beginners

Why do we need classes?

By now you have written programs with variables, lists, and functions. That works well for small tasks, but things get awkward as soon as you need to keep track of several pieces of related information at once. Imagine a program about dogs. Each dog has a name and an age. With what you know so far, you might write something like this:

```
dog1_name = "Buddy"
dog1_age = 3
dog2_name = "Luna"
dog2_age = 5
```

This is already clumsy with two dogs, and it falls apart with twenty. Nothing in the code says that `dog1_name` and `dog1_age` belong together; you just have to remember. And if you write a function that works with a dog, you have to pass the name and the age separately every time.

A **class** solves this. It lets you describe a new kind of thing (a Dog) once, and then create as many individual dogs as you like, each one carrying its own name and age. You have actually been using classes all along: `str`, `list`, and `dict` are all classes built into Python. When you write `"hello".upper()`, you are calling a method on a string object. The goal of this guide is to show you how to make your own.

Three words to know

The whole topic rests on three ideas. Everything else is detail.

Class	A blueprint or template. It describes what every object of that kind will have and what it can do. Example: the idea of a *Dog* in general.
Object	One concrete thing built from the blueprint. Also called an *instance*. Example: Buddy, a specific dog who is 3 years old. You can create many objects from one class.
Method	A function that lives inside a class and works on one object at a time. Example: a *bark* action that every dog can perform.

A useful comparison: a cookie cutter is a class, and each cookie is an object. The cutter decides the shape; each cookie is a separate thing you can pick up, decorate, or eat without affecting the others.

> **Function or method?** Both are defined with `def`. The difference is where they live and how you call them. A function stands alone and you call it by name: `len(my_list)`. A method belongs to a class and you call it on an object with a dot: `my_list.append(4)`. When you see a dot followed by parentheses, you are looking at a method call.

Your first class

Here is a complete Dog class and two dogs made from it. Read it once, then we will go through it line by line.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

buddy = Dog("Buddy", 3)
luna = Dog("Luna", 5)

print(buddy.name)
print(luna.age)
```

OUTPUT

```
Buddy
5
```

`class Dog:` starts the definition. Class names are written in CapitalCase by convention, so Python programmers can tell at a glance that Dog is a class and not a variable or a function. Everything indented under this line belongs to the class.

`def __init__(self, name, age):` defines a special method that Python runs automatically every time a new object is created. The name is short for *initialize*, with two underscores on each side. Its job is to set up the new object. You never call `__init__` yourself; writing `Dog("Buddy", 3)` calls it for you.

`self.name = name` stores the name *inside* the object. Values attached to an object like this are called **attributes**. Without `self.` the name would be an ordinary local variable that disappears when the method ends. With it, the value stays with the object for as long as the object exists.

`buddy = Dog("Buddy", 3)` creates an object. Python makes a new, empty Dog, passes it to `__init__` as `self`, and passes `"Buddy"` and `3` as `name` and `age`. The finished object is stored in the variable `buddy`.

`buddy.name` reads an attribute back out. The dot means "look inside this object". Notice that `buddy` and `luna` have their own separate values. Changing one never affects the other.

> **What is self?** `self` is the object currently being worked on. When you call `buddy.bark()`, Python quietly translates it to `Dog.bark(buddy)`, so inside the method `self` *is* `buddy`. The word `self` is only a convention, but it is one that every Python programmer follows, so please use it too. It must be the first parameter of every method, and you never pass it yourself.

Adding methods

So far our dogs only store data. Methods let them *do* things. A method is defined exactly like a function, except that it is indented inside the class and its first parameter is `self`. From here on, the examples show only the new methods; picture them written inside the Dog class from the previous page, at the same indentation as `__init__`.

ADDED INSIDE THE DOG CLASS, BELOW __init__

```
    def bark(self):
        print(self.name + " says woof!")

    def have_birthday(self):
        self.age = self.age + 1

buddy = Dog("Buddy", 3)
buddy.bark()
buddy.have_birthday()
print(buddy.age)
```

OUTPUT

```
Buddy says woof!
4
```

`bark` reads an attribute and uses it. `have_birthday` goes further and *changes* one. Because the change is made through `self`, it is permanent: the next time you look at `buddy.age` it is 4, not 3. This is the heart of why objects are useful. An object remembers its own state, and its methods are the approved ways of reading and changing it.

Methods can also take arguments and return values, like ordinary functions. Here are two more: one returns a computed value, the other accepts a second dog as an argument.

ADDED INSIDE THE DOG CLASS, BELOW have_birthday

```
    def human_years(self):
        return self.age * 7

    def is_older_than(self, other_dog):
        return self.age > other_dog.age

buddy = Dog("Buddy", 3)
luna = Dog("Luna", 5)
print(buddy.human_years())
print(luna.is_older_than(buddy))
```

OUTPUT

```
21
True
```

Look carefully at `luna.is_older_than(buddy)`. The object before the dot becomes `self` and the one in the parentheses becomes `other_dog`, so `self.age` is 5 and `other_dog.age` is 3. The object before the dot is always passed in first.

Printing an object nicely

If you try `print(buddy)`, you get something like `<__main__.Dog object at 0x7f3a...>`: the type and a memory address, because Python has no idea how *you* want a dog displayed. Fix it with a special method called `__str__` that returns a string. Methods with double underscores on both sides, like `__init__` and `__str__`, are nicknamed *dunder* methods, and Python calls them for you at the right moment.

ADDED INSIDE THE DOG CLASS

```
    def __str__(self):
        return f"{self.name} ({self.age} years old)"
buddy = Dog("Buddy", 3)
print(buddy)
```

OUTPUT

```
Buddy (3 years old)
```

A complete example: a bank account

Let's put everything together in a second class, this time with a little logic inside the methods. A bank account has an owner and a balance. You can deposit money, and you can withdraw money as long as there is enough in the account.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance = self.balance + amount
        print(f"Deposited {amount}. New balance: {self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Not enough money.")
        else:
            self.balance = self.balance - amount
            print(f"Withdrew {amount}. New balance: {self.balance}")

alice = BankAccount("Alice", 100)
bob = BankAccount("Bob")

alice.deposit(50)
bob.withdraw(20)
alice.withdraw(30)
print(bob.balance)
```

OUTPUT

```
Deposited 50. New balance: 150
Not enough money.
Withdrew 30. New balance: 120
0
```

A few things worth noticing. The parameter `balance=0` has a default value, so `BankAccount("Bob")` creates an account with nothing in it, while `BankAccount("Alice", 100)` starts at 100. This works exactly the way defaults work in ordinary functions.

Alice and Bob are completely independent. Alice's deposit does not touch Bob's balance, and Bob's failed withdrawal does not touch Alice's. Each object carries its own `balance` attribute, and each method call works only on the object before the dot.

The `withdraw` method protects the account. Because the only way to take money out is through this method, and the method checks the balance first, an account can never go negative by accident. Bundling data together with the rules for changing it is the main reason programmers reach for classes.

Common mistakes

Almost everyone makes these when starting out, so it is worth seeing them once in a controlled setting.

Forgetting `self` in the method definition	Writing `def bark():` instead of `def bark(self):` produces the error *bark() takes 0 positional arguments but 1 was given*. Python passed the object in automatically, but there was no parameter to receive it.
Forgetting `self.` when storing a value	Writing `name = name` inside `__init__` creates a local variable that vanishes immediately. Later, `buddy.name` fails with *'Dog' object has no attribute 'name'*. If you want it to stick to the object, it needs `self.` in front.
Forgetting the parentheses when calling	`buddy.bark` without parentheses does not run the method. It just refers to it, and nothing happens. Methods are called with `()`, the same as functions. Attributes like `buddy.name` are read without parentheses.
Using the class when you meant an object	`Dog.bark()` fails because there is no particular dog to bark. You must first create an object with `Dog(...)` and call the method on that. The class is the recipe; you cannot eat the recipe.

Try it yourself

Reading about classes is not the same as writing them. Each exercise below should take only a few minutes, and each one builds on the last.

1. Write a `Rectangle` class with `width` and `height` attributes and an `area()` method that returns width times height. Create two rectangles and print both areas.

2. Add a `perimeter()` method and a `__str__` method so that printing a rectangle shows something like `Rectangle 4 x 5`.

3. Write a `Student` class with a `name` and an empty list of `grades`. Add an `add_grade(grade)` method and an `average()` method. Watch out: what should `average()` return when there are no grades yet?

4. Go back to `BankAccount` and add a `transfer(amount, other_account)` method that moves money from one account to another, but only if there is enough. This is the same pattern as `is_older_than`: one object before the dot, another one inside the parentheses.

Summary

A **class** is a blueprint written with `class Name:`. An **object** is one concrete thing made from it by calling the class like a function: `Name(...)`. **Attributes** are values stored on an object using `self.attribute = value`, usually inside `__init__`. **Methods** are functions defined inside the class whose first parameter is `self`; you call them on an object with a dot, and `self` automatically becomes that object. Every object has its own copy of its attributes, so changing one object never changes another. That is the core idea. Everything else in object-oriented programming, including inheritance, is built on top of these few rules.